[Institution name]
[Master's programme]

Oracle Database Administration

Project 22

Oracle Database Performance Optimization Through Partitioning and Data Lifecycle Management

---

Technical Report



Team Mohamed Baounna
[Second team member]

Supervisor [Supervising professor]

Academic year 2026–2027

Repository `https://github.com/Baounna/oracle-billing-lifecycle`

Abstract

A mobile telecom operator keeps six years of billing records in a single Oracle table. Monthly and audit reports slow down as the table grows, purging old data is a long `DELETE` that produces gigabytes of redo, and the legal retention rules are applied by hand. As two consultants, we were asked to design and evaluate a data lifecycle strategy that combines partitioning, indexing, compression and archiving, and to prove its effect with measurements.

We built the solution on Oracle Database 19c Enterprise Edition (in Docker) and compared it with the client's current design on the same synthetic dataset of 23.76 million billing rows over 72 months. The optimised design partitions the billing table by month with interval partitioning. A PL/SQL package (`DLM_PKG`) moves every month through four tiers. Hot months stay uncompressed and writable. Warm months are compressed and read-only. Archived months are moved to an archive table by two partition exchanges, without copying a row. Months past the 60-month legal retention are dropped. Each operation is guarded, logged, resumable after a crash, and validated against control totals captured when the data was loaded.

Measured results: the month-end revenue report reads 54 times fewer blocks and runs 14 times faster; an audit query on an archived month is 43 times faster. Archiving one month takes 0.42 s instead of 26 s, and purging one takes 0.02 s instead of 5.1 s. The full monthly job, including the compression step, is 8.9 times faster and produces 3.8 times less redo. Retained data uses 44 % less storage. All 62 retained months match their control totals. The solution passes 43 automated functional tests and 25 security checks.

Keywords: Oracle Database 19c, interval partitioning, partition exchange, data lifecycle management, archiving, basic table compression, partition pruning, retention policy, PL/SQL, performance benchmarking.

Contents

Abstract **i**

1 General Introduction **1**
1.1 Context . . . 1
1.2 Motivation . . . 1
1.3 Problem statement . . . 1
1.4 Objectives . . . 1
1.5 Structure of the report . . . 2

2 Client Company Presentation **3**
2.1 Business sector and company . . . 3
2.2 IT environment . . . 3
2.3 Project sponsor . . . 3
2.4 Business requirements . . . 3
2.5 Operational constraints . . . 4

3 Requirements Analysis and Project Specifications **5**
3.1 Functional requirements . . . 5
3.2 Technical and non-functional requirements . . . 5
3.3 Retention policy . . . 5
3.4 Measurable indicators and acceptance criteria . . . 6

4 Technology Assessment and Justification **7**
4.1 Database platform . . . 7
4.2 Data segmentation . . . 7
4.3 Archiving method . . . 8
4.4 Compression and licensing . . . 8
4.5 Tooling . . . 9

5 System Design and Architecture **10**
5.1 Overall architecture . . . 10
5.2 Storage tiers and data lifecycle . . . 10
5.3 Archive sequence . . . 12
5.4 Monthly job (activity) . . . 14
5.5 Logical data model . . . 15
5.6 Indexing strategy . . . 16
5.7 Security architecture . . . 16

6 Implementation Environment **18**
6.1 Hardware and software . . . 18
6.2 Storage configuration . . . 18
6.3 Prerequisites and installation . . . 20

7 Solution Implementation **21**
7.1 Repository organisation . . . 21
7.2 Data generation . . . 21
7.3 Partitioned schema . . . 21
7.4 The lifecycle package `DLM_PKG` . . . 22
7.4.1 WARM: freeze, capture, move . . . 22
7.4.2 ARCHIVE: two exchanges, resumable . . . 22
7.4.3 PURGE and guards . . . 23
7.4.4 Integrity validation . . . 23
7.5 Application write window . . . 23
7.6 Baseline package . . . 24
7.7 Benchmark harness . . . 24

8 Practical Usage Examples **25**
8.1 Scenario 1 – State of the tiers . . . 25
8.2 Scenario 2 – Month-end revenue report (business scenario) . . . 25
8.3 Scenario 3 – Dry run of the monthly job . . . 26
8.4 Scenario 4 – Incident: archive job crashes, then recovers . . . 26
8.5 Scenario 5 – Guard and monthly job . . . 27
8.6 Scenario 6 – What the billing application cannot do . . . 27

9 Testing and Validation **28**
9.1 Methodology . . . 28
9.2 Functional test scenarios . . . 28
9.3 Security test scenarios . . . 29
9.4 Verification of the acceptance criteria . . . 29

10 Performance and Reliability Evaluation **30**
10.1 Measurement protocol . . . 30
10.2 Query performance . . . 30
10.3 Maintenance operations . . . 32
10.4 Storage . . . 33
10.5 Integrity and reliability . . . 33
10.6 Indicators against targets . . . 33

11 Challenges Encountered and Solutions **35**

12 Discussion and Future Improvements **37**
12.1 Strengths 37
12.2 Limitations and remaining risks 37
12.3 Possible improvements 37

13 General Conclusion **39**

References **40**

A Team Organisation and Schedule **41**
A.1 Four-week plan 41
A.2 Distribution of responsibilities 41

B Lifecycle Package Interface **42**

C Test Output (excerpt) **43**

D Reproducing the Results **44**

List of Figures

5.1 Overall system architecture 11
5.2 Storage tiers and lifecycle operations 12
5.3 Archive of one month: sequence diagram 13
5.4 Monthly lifecycle job: activity diagram 14
5.5 Logical data model 15

6.1 Deployment diagram 19

10.1 Logical reads per query on the full history (log scale) 31
10.2 Median elapsed time per query after the retention policy (log scale) 31
10.3 Maintenance time per operation (log scale) 32

List of Tables

2.1 Operational constraints 4

3.1 Functional requirements 5
3.2 Technical requirements 5
3.3 Retention policy parameters 6
3.4 Key performance indicators and targets 6

4.1 Database platform options 7
4.2 Segmentation options 7
4.3 Archiving options 8
4.4 Compression options 8
4.5 Tooling choices 9

5.1 Indexes of both designs 16
5.2 Accounts and privileges (least privilege) 16

6.1 Environment and versions 18
6.2 Tablespaces and size caps 18

9.1 Functional tests (all 43 pass) 28
9.2 Security checks (all 25 pass) 29
9.3 Acceptance criteria 29

10.1 Workload results on the full 72-month history (before lifecycle), medians 30
10.2 Workload results after the retention policy (Q7 through the online + archive views) 30
10.3 Maintenance cost per operation (averages) 32
10.4 Space used, tables and their indexes (MB) 33
10.5 KPI results 33

11.1 Main problems and corrective actions 35

A.1 Project schedule 41
A.2 Responsibilities 41

B.1 Error codes raised by the lifecycle code 42

1 General Introduction

1.1 Context

Billing is one of the largest data sets of a telecom operator. Each call, SMS, data session and roaming event becomes a billed line, and those lines must be kept for years for customer care, finance, disputes and legal audits. In many companies this history accumulates in the same table that the billing application writes to every day. The table grows without limit, and work that was cheap when it held one year of data becomes slow with six.

This project was carried out within the Oracle Database Administration course. Its brief (Project 22) asks for a data lifecycle strategy that combines partitioning where it is licensed, indexing, archiving and performance benchmarking, and for proof of its effect on comparable workloads.

1.2 Motivation

Three symptoms of the client's current design drive the work:

- **Reports slow down every month.** A report on last month's revenue reads the whole six-year table, because one month is too large a share of the rows for an index to help.
- **Retention is expensive.** Removing a month means `DELETE`-ing hundreds of thousands of rows. Every row and every index entry generates undo and redo, and the table does not even shrink afterwards.
- **Retention is manual and risky.** Nothing prevents someone from deleting data that is still under legal retention, and nothing proves afterwards that no billing record was lost or changed.

1.3 Problem statement

How can the billing history be organised so that frequent reports read only the period they need, old data moves to cheaper storage and is removed at the end of its retention with near-zero cost, while guaranteeing, and proving, that no billing record is lost or altered?

1.4 Objectives

1. Generate a representative, reproducible billing history (tens of millions of rows).
2. Measure the performance of the current design on a defined workload: the *baseline*.
3. Design a partitioned schema and a lifecycle policy (hot, warm, archive, purge) suited to the client's retention rules and licences.
4. Implement the lifecycle as automated, guarded, logged and recoverable database code.
5. Measure query, maintenance and storage effects on the same workload, and verify data integrity objectively.

6. Document the licensing restrictions, trade-offs and limits of the solution.

1.5 Structure of the report

Chapter 2 presents the client. Chapters 3 and 4 analyse the requirements and compare the possible technologies. Chapter 5 describes the architecture, and Chapters 6 and 7 the environment and the implementation. Chapter 8 shows practical scenarios. Chapters 9 and 10 present the tests and the measured results. Chapters 11 to 13 discuss the difficulties, the limits and the outcome. All the code, test outputs and benchmark data are in the project repository (`https://github.com/Baounna/oracle-billing-lifecycle`).

2 Client Company Presentation

2.1 Business sector and company

The client is a mobile telecom operator (a fictitious company created for this scenario) serving about 500 000 subscribers in twelve regions. It sells prepaid, postpaid and business plans (twenty tariff plans) and bills four kinds of service: voice, SMS, mobile data and roaming. About 10 % of the customers are businesses.

2.2 IT environment

Billing is handled by an in-house application that writes billing lines into an Oracle database. Finance, customer care, collections and the legal department query the same data. The database team runs Oracle Database 19c Enterprise Edition, which already includes the Partitioning option in the client's licence. For this assignment, the database is a 19c container (Oracle 19.19, ARM64) running under Docker on a team member's laptop, with a single pluggable database `ORCLPDB1`.

2.3 Project sponsor

The project is sponsored by the **DBA Manager**. They are responsible for the performance of the billing database, the night maintenance window, and compliance with the retention rules defined by the legal department.

2.4 Business requirements

- Month-end finance reports and customer-care screens must stay fast even as history grows.
- Billing data must be kept for **60 months** (legal retention) and then deleted.
- Data older than three years is rarely read but must stay queryable for audits and disputes.
- The billing application must never be blocked, and must never be able to change or delete history.
- Every retention action must be traceable and provably safe.

2.5 Operational constraints

Table 2.1: Operational constraints

Constraint	Description
Availability	Lifecycle operations must not stop the billing application (partition-level work, short locks).
Security	Least privilege, no credentials in code, read-only history, audit trail.
Performance	Reports filtered on dates must read only the requested period.
Licensing	Use only features covered by Enterprise Edition + Partitioning; no Advanced Compression, no Diagnostics/Tuning Packs.
Resources	Laptop with Docker: 8 vCPU and about 8 GB of RAM for Docker; the disk was nearly full (about 20 GB free at the start).
Schedule	Four weeks: design, implementation, tests and measurements, deliverables.
Data	Synthetic, anonymised data only.

3 Requirements Analysis and Project Specifications

3.1 Functional requirements

Table 3.1: Functional requirements

ID	Requirement
F1	Store the billing history so that monthly and annual reports read only the period requested.
F2	Apply the retention policy automatically: hot (12 months), online (36 months), archive (until 60 months), purge after.
F3	Keep archived data queryable (audit, legal requests) through a single access point.
F4	Allow the retention parameters to change without modifying code.
F5	Provide a dry-run mode that shows what the job would do.
F6	Log every lifecycle action: what, when, how many rows, how long, result.
F7	Prove the integrity of billing data after every operation.

3.2 Technical and non-functional requirements

Table 3.2: Technical requirements

ID	Requirement
T1	Lifecycle operations must avoid copying or deleting rows one by one where possible.
T2	A failure in the middle of an operation must not lose data, and a re-run must recover.
T3	Data inside the legal retention can never be purged, whatever parameter is passed.
T4	Old billing data cannot be modified (by accidental updates or late inserts).
T5	Least privilege: the billing application cannot delete history or run lifecycle jobs, and analysts are read-only.
T6	No credentials in source code or in the Git repository.
T7	The whole solution must be reproducible from scripts on the course Docker environment.
T8	The lab must not exhaust the laptop disk.
T9	Use only licensed features, and document the licence impact.

3.3 Retention policy

The legal and business rules translate into four tiers, stored as parameters in the `DLM_CONFIG` table rather than hard-coded:

Table 3.3: Retention policy parameters

Parameter	Value	Meaning
`HOT_MONTHS`	12	months kept uncompressed and writable
`ONLINE_MONTHS`	36	months kept in the online billing table (hot + warm)
`RETENTION_MONTHS`	60	legal retention; older months are purged
`MIN_RETENTION_MONTHS`	60	safety floor: no purge may use a shorter retention
`AS_OF_MONTH`	2026-11	reference "today", so that runs are reproducible

3.4 Measurable indicators and acceptance criteria

We formalised the indicators after a first exploratory benchmark and before the final run whose numbers this report presents. The brief asks for indicators to be defined before testing, so we state this order openly. The indicators compare the two designs on the same data, with the same SQL text and bind values.

Table 3.4: Key performance indicators and targets

KPI	Indicator	Target
K1	Month-end report (Q1): elapsed time and logical reads	$\geq$5$\times$ time, $\geq$10$\times$ LIO
K2	Audit query on an archived month (Q7)	$\geq$5$\times$ fewer LIO
K3	OLTP lookups (Q2 by customer, Q5 by id)	no time regression $>$2$\times$
K4	Archive one month (and the whole monthly job)	$\geq$10$\times$ faster, $<$5 s
K5	Purge one month	$<$1 s
K6	Monthly load + statistics refresh	not slower than baseline
K7	Storage of the retained data	$\geq$25 % smaller
K8	Data integrity vs load-time control totals	100 % of months
K9	Recovery after a crash	0 rows lost, recovery = re-run
K10	Security controls (asserted checks)	100 %
K11	Automation	one command per month

The acceptance criteria of the brief are: (1) the design preserves data integrity, (2) retention operations work correctly, (3) query or maintenance improvements are measured on comparable workloads, and (4) licensing restrictions and technical trade-offs are documented. Chapter 10 checks each of them.

4 Technology Assessment and Justification

4.1 Database platform

The brief allows Oracle Database Free *if supported by the selected version* or a licensed 19c when required. We checked the course container before choosing: `V$OPTION` reports `Partitioning = TRUE`, and a test interval-partitioned table could be created and dropped.

Table 4.1: Database platform options

Option	Advantages	Drawbacks	Decision
Oracle Database 23ai Free	free, recent, partitioning available	12 GB user data, 2 GB RAM, not the course environment	not needed
Oracle 19c EE (course container)	installed, Partitioning option present, long-term support release used by most companies	EE and Partitioning are paid options in production	**chosen**

4.2 Data segmentation

Table 4.2: Segmentation options

Option	Advantages	Drawbacks	Decision
Single heap table + indexes (current)	simple	full scans for period reports; retention by `DELETE`	baseline
One table per month + `UNION ALL` view	works without the Partitioning option	views to regenerate monthly, inserts must be routed, weaker pruning with binds	documented alternative
Range partitioning with a job creating partitions	standard	inserts fail if the job is late	rejected
Interval-range partitioning by month on `bill_date`	partitions created automatically, pruning, partition-level maintenance	Partitioning option, system-generated names	**chosen**
Composite range-list (by service)	finer pruning on service	more segments; the workload does not filter on service alone	rejected

A **month** is the right partition unit because the retention rule, the billing cycle and most reports are monthly. One month holds about 230 000 to 430 000 rows (20–35 MB), so each partition can

be scanned, moved, exchanged or dropped in seconds.

4.3 Archiving method

Table 4.3: Archiving options

Option	Advantages	Drawbacks	Decision
`INSERT ... SELECT` + `DELETE` (current)	simple	copies and deletes every row; large redo and undo; table does not shrink	baseline
Data Pump export of old partitions + drop	data leaves the database	no longer queryable in SQL; restore needed for audits	rejected (F3)
Exchange partition into an archive table	metadata operation; archive stays queryable	global index maintenance; identical structures required	**chosen**

4.4 Compression and licensing

Table 4.4: Compression options

Option	Licence	Decision
BASIC (`ROW STORE COMPRESS BASIC`)	included in Enterprise Edition (not available in SE2)	**chosen** for read-only data
ADVANCED (OLTP)	Advanced Compression option	not used (licence)
Hybrid Columnar Compression	Exadata or engineered storage only	not available

BASIC compression only compresses direct-path operations, and conventional updates on compressed blocks are inefficient. That fits warm and archived data, which is written once by `MOVE PARTITION` and then made read-only. The measurement method relies only on base-product views (`V$MYSTAT`, `V$SQL`, `DBMS_XPLAN.DISPLAY_CURSOR`). We deliberately did not use AWR, ASH or SQL Monitor, which need the Diagnostics and Tuning Packs.

4.5 Tooling

Table 4.5: Tooling choices

Need	Choice	Reason
Lifecycle logic	PL/SQL package	runs next to the data, transactional logging, schedulable
Data generation	set-based SQL with `ORA_HASH`	millions of rows per minute, deterministic
Benchmark harness	Python 3.12 + python-oracledb	session statistics, plans, JSON, charts
SQL clients	SQL*Plus (DB container), SQLcl (tools)	nothing installed on the host
Automation	Bash scripts + Docker Compose	reproducible on any Docker host
Demonstration interface	Streamlit web dashboard (tools image)	every scenario by clicks, no terminal; calls the same package
Version control	Git, GitHub	required by the brief

5 System Design and Architecture

5.1 Overall architecture

Figure 5.1 shows the components and their interactions. Three kinds of clients use the database, each through a different account and access path:

- The **billing application** (`TELCO_APP`) writes and reads through the view `BILLING_CURRENT_V`. The view exposes only the hot months and rejects any row outside them.
- **Analysts and auditors** (`TELCO_ANALYST`) read the whole retained history through `BILLING_ALL`, the union of the online and archive tables.
- The **DBA or scheduler** (`TELCO_OWNER`) runs the lifecycle package `DLM_PKG`. The package reads its policy from `DLM_CONFIG`, logs to `DLM_JOB_LOG` and validates against `BILLING_CONTROL_TOTALS`.

The benchmark harness runs in a separate tools container and reaches the database over TCP. The baseline tables (`BILLING_RECORDS_BASE`, `BILLING_ARCHIVE_BASE`) live in the same schema, so both designs are compared on the same server, data and workload.

5.2 Storage tiers and data lifecycle

Each month moves through four states (Figure 5.2). Each state has its own tablespace, storage format and access rights. Tablespaces separate the tiers physically: `TELCO_HOT` for current data, `TELCO_IDX` for its indexes, and `TELCO_ARCH` for compressed warm and archived data. In production, `TELCO_ARCH` could sit on cheaper storage.

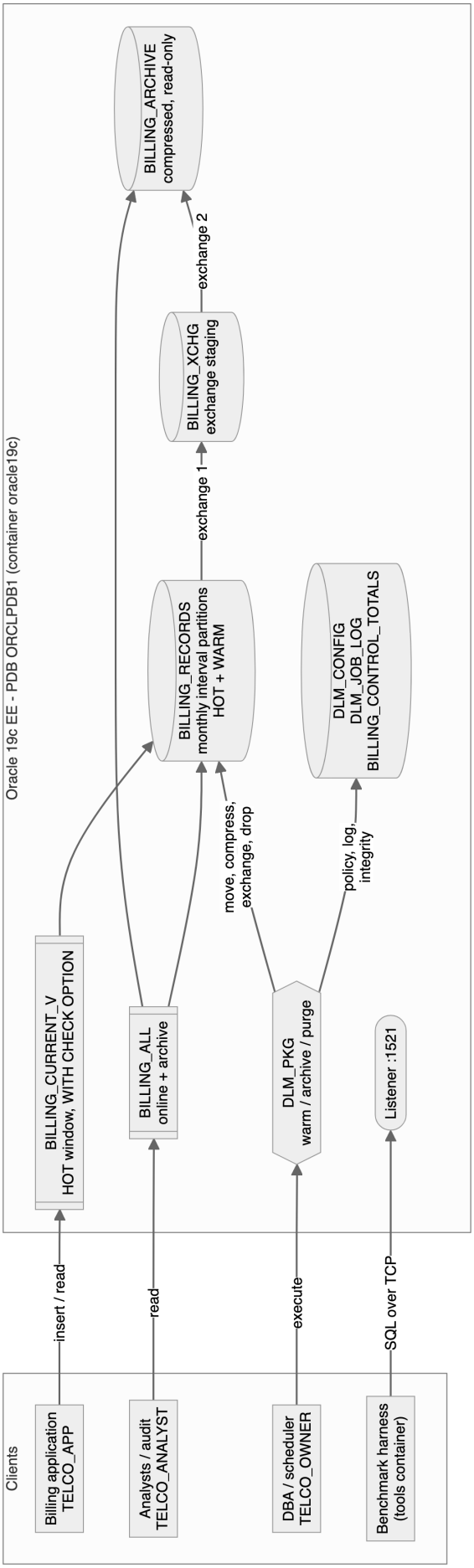


Figure 5.1: Overall system architecture

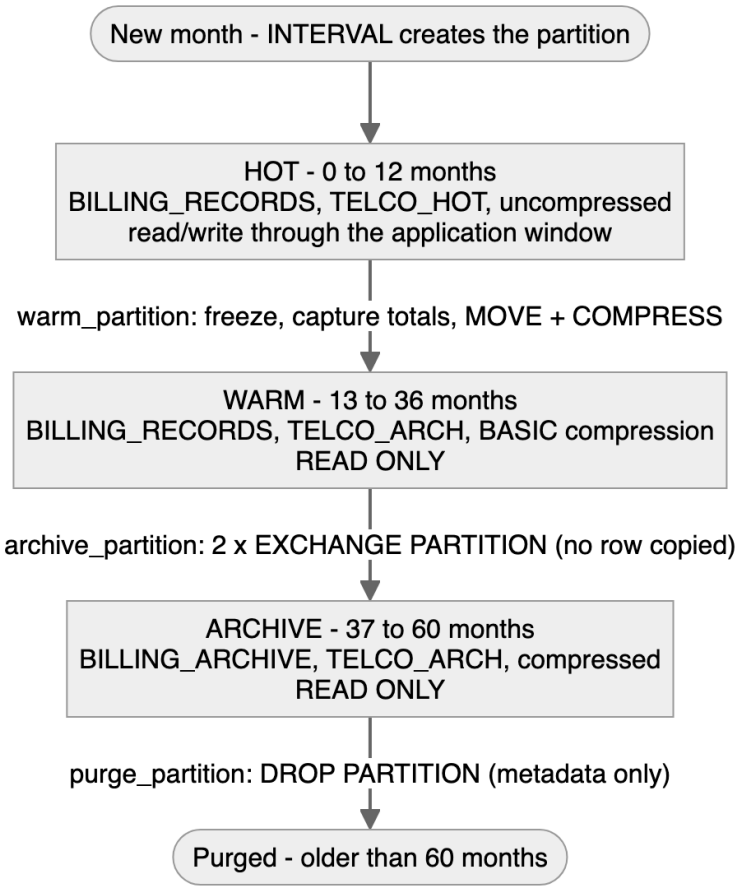


Figure 5.2: Storage tiers and lifecycle operations

HOT (0–12 months)
Uncompressed in `TELCO_HOT`. The application may insert late lines and corrections here through its window view.

WARM (13–36 months)
The month is *frozen*: it is set read-only, its control totals are re-captured, and it is moved to `TELCO_ARCH` with BASIC compression. It stays in `BILLING_RECORDS`, so recent-history queries use the same table.

ARCHIVE (37–60 months)
The month leaves the online table for `BILLING_ARCHIVE` through two partition exchanges, and stays compressed and read-only.

PURGED (>60 months)
The archive partition is dropped, which is a metadata operation.

5.3 Archive sequence

The key mechanism is the zero-copy archive (Figure 5.3). A partition can only be exchanged with a non-partitioned table, so the month travels through the empty staging table `BILLING_XCHG`. Step 1 swaps the month's segment into the staging table. Step 2 creates the archive partition (`LOCK TABLE ... PARTITION FOR`) and swaps the segment into it. Step 3 drops the now-empty online partition. No row is copied at any point. If the job dies between steps 1 and 2, the data waits in `BILLING_XCHG`: the integrity check reports the month as missing, and the next run detects the staged rows and resumes at step 2.

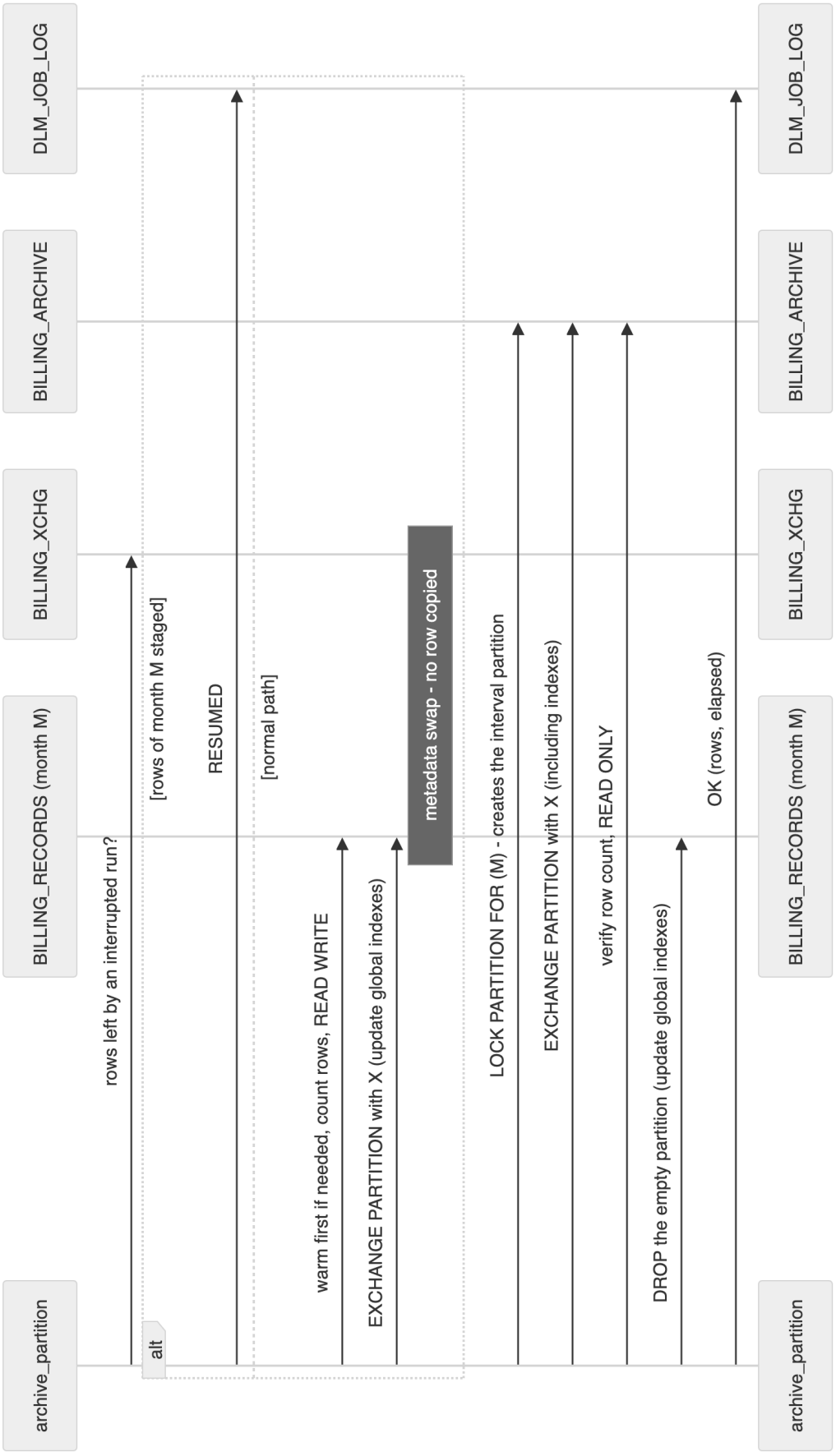


Figure 5.3: Archive of one month: sequence diagram

5.4 Monthly job (activity)

`run_lifecycle` orchestrates a complete monthly run (Figure 5.4). It first closes the application window on the months it is about to freeze, then resumes any interrupted archive, removes empty partitions left by rejected inserts, and re-applies read-only on archive partitions. It then purges, warms and archives, refreshes incremental statistics and validates every frozen month. If a frozen month differs from its control totals, the job stops with `ORA-20040` so that an operator investigates.

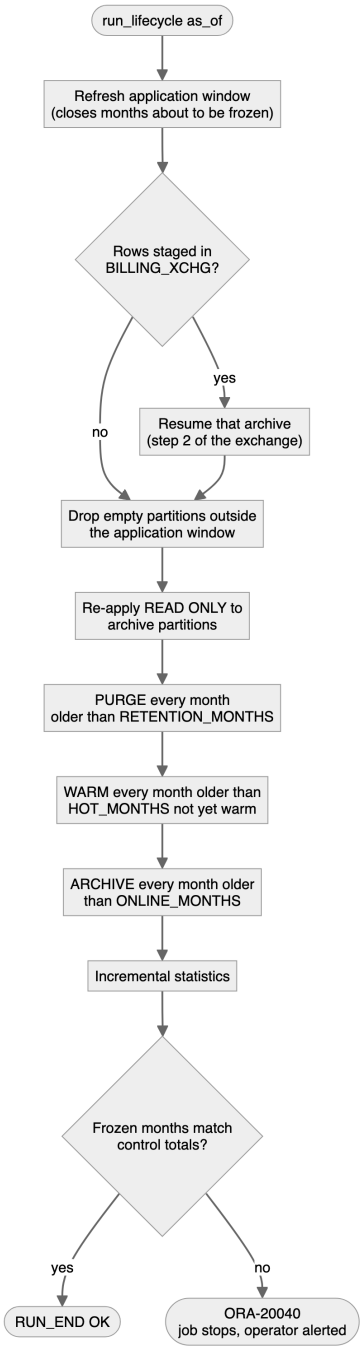


Figure 5.4: Monthly lifecycle job: activity diagram

5.5 Logical data model

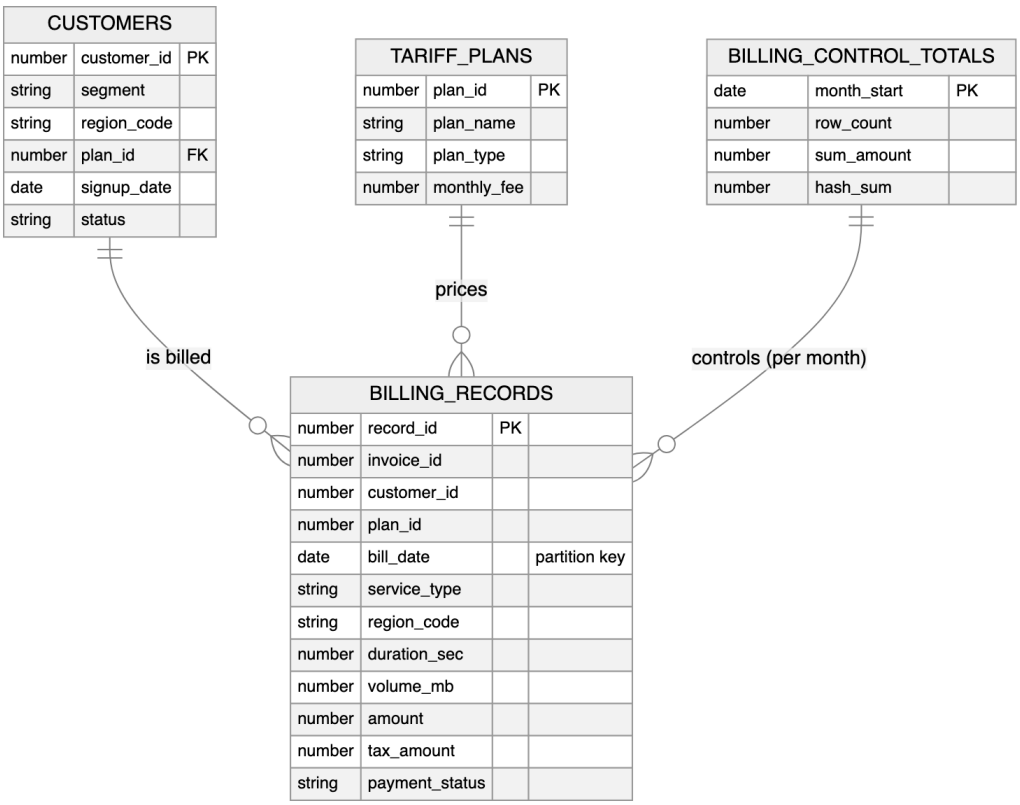


Figure 5.5: Logical data model

`BILLING_RECORDS` is the fact table (one row per billed service line). Each customer subscribes to one tariff plan (`CUSTOMERS.plan_id`, foreign key), and the plan is copied on every billing line. The other billing tables (`BILLING_ARCHIVE`, `BILLING_XCHG`, the baseline tables and the load staging table) are created from `BILLING_RECORDS_BASE` with `CREATE TABLE ... AS SELECT ... WHERE 1=0`. This guarantees identical columns, which partition exchange requires. The same `CHECK` constraints (service type, payment status, non-negative amounts) are added to every billing table. There is no foreign key from the fact table to `CUSTOMERS`: as in most billing and warehouse designs, the loading process guarantees referential integrity, and a foreign key would complicate exchanges and slow bulk loads.

5.6 Indexing strategy

Table 5.1: Indexes of both designs

Design	Columns	Type	Purpose
Baseline	`record_id`	PK, B-tree	OLTP lookup
Baseline	`(customer_id, bill_date)`	B-tree	customer history
Baseline	`bill_date`	B-tree	date-range reports
Partitioned	`record_id`	PK, **global** B-tree	single probe for any online month
Partitioned	`(customer_id, bill_date)`	**local** B-tree	one segment per month, moves with it
Partitioned	*none on* `bill_date`	–	replaced by partition pruning
Archive	`(customer_id, bill_date)`	local B-tree	swapped in by `INCLUDING INDEXES`

The main trade-off is the global primary key. It keeps lookups by `record_id` to a single probe, but it must be maintained (`UPDATE GLOBAL INDEXES`) whenever a partition is moved or exchanged out. A local unique index must contain the partition key (ORA-14039), so the alternative would be a local primary key on `(record_id, bill_date)`. Lifecycle operations would then be pure metadata, but `record_id` alone would no longer be guaranteed unique, and lookups by `record_id` would probe every partition.

5.7 Security architecture

Table 5.2: Accounts and privileges (least privilege)

Account	Privileges	Cannot
`TELCO_OWNER`	owns the objects, runs DLM, `SELECT` on a few `V$` views for benchmarking	– (would be locked in production, with the job run by a scheduler)
`TELCO_APP`	`INSERT`, `READ` on `BILLING_CURRENT_V`; `READ` on reference data	write outside the hot window, touch base tables, update or delete, read the archive or the policy, run DLM, DDL
`TELCO_ANALYST`	`READ` on `BILLING_ALL` and reference data	modify anything, lock tables, access base tables, policy or DLM

The other security measures are:

- **No credentials in code or in Git.** A git-ignored `.env` (mode 600) is generated with random passwords. Passwords are sent to SQL*Plus through standard input, so they never appear in the process list. They are only used in `CREATE/ALTER USER` statements, which Oracle does not keep in clear text in `V$SQL`.
- **`READ` instead of `SELECT`** for read-only accounts. With `SELECT`, an account can `LOCK TABLE` a view and block the lifecycle job; we verified this on the database. `READ` prevents it.

- **Frozen data is read-only** (ORA-14466 on any change). The application window rejects late rows in old months (ORA-01402).
- **Guards and audit trail.** Purges inside the legal retention are refused, and every action, refusal and error is logged.

6 Implementation Environment

6.1 Hardware and software

Figure 6.1 shows where each component runs. The database and the tools run in two containers on the same laptop; the scripts drive SQL*Plus inside the database container and the benchmark connects over TCP.

Table 6.1: Environment and versions

Component	Version / configuration
Host	Apple silicon laptop, macOS, Docker Desktop 29.0 (8 vCPU, about 8 GB RAM for the Docker VM)
Database	Oracle Database 19c Enterprise Edition 19.19 (ARM64), container `oracle19c`, CDB `ORCLCDB`, PDB `ORCLPDB1`
Memory	SGA 2.3 GB (buffer cache 1.7 GB), PGA target 784 MB
Options	Partitioning (used), Advanced Compression (not used), Diagnostics/Tuning Packs (not used)
Tools container	Python 3.12, python-oracledb 2.5.1 (thin), pandas 2.2, matplotlib 3.9, Streamlit 1.40, SQLcl 26.3, OpenJDK 17
Clients	SQL*Plus inside the database container (OS authentication for SYSDBA)
Version control	Git, GitHub (`https://github.com/Baounna/oracle-billing-lifecycle`)

6.2 Storage configuration

The laptop disk had only about 20 GB free when the project started, and the defaults of the course container let UNDO and TEMP grow to 32 GB each. Before generating any data we capped every file that could grow (Table 6.2). We also freed about 9 GB by removing unused Docker images. Every data-loading script checks the free host disk first and stops below a threshold.

Table 6.2: Tablespaces and size caps

Tablespace	Cap	Content
`TELCO_HOT`	7 GB	hot partitions, baseline table, reference data
`TELCO_IDX`	4 GB	indexes of hot data
`TELCO_ARCH`	4 GB	warm and archived partitions (compressed) and their indexes, baseline archive
`UNDOTBS1`	4 GB	capped (default 32 GB)
`TEMP`	3 GB	capped (default 32 GB)

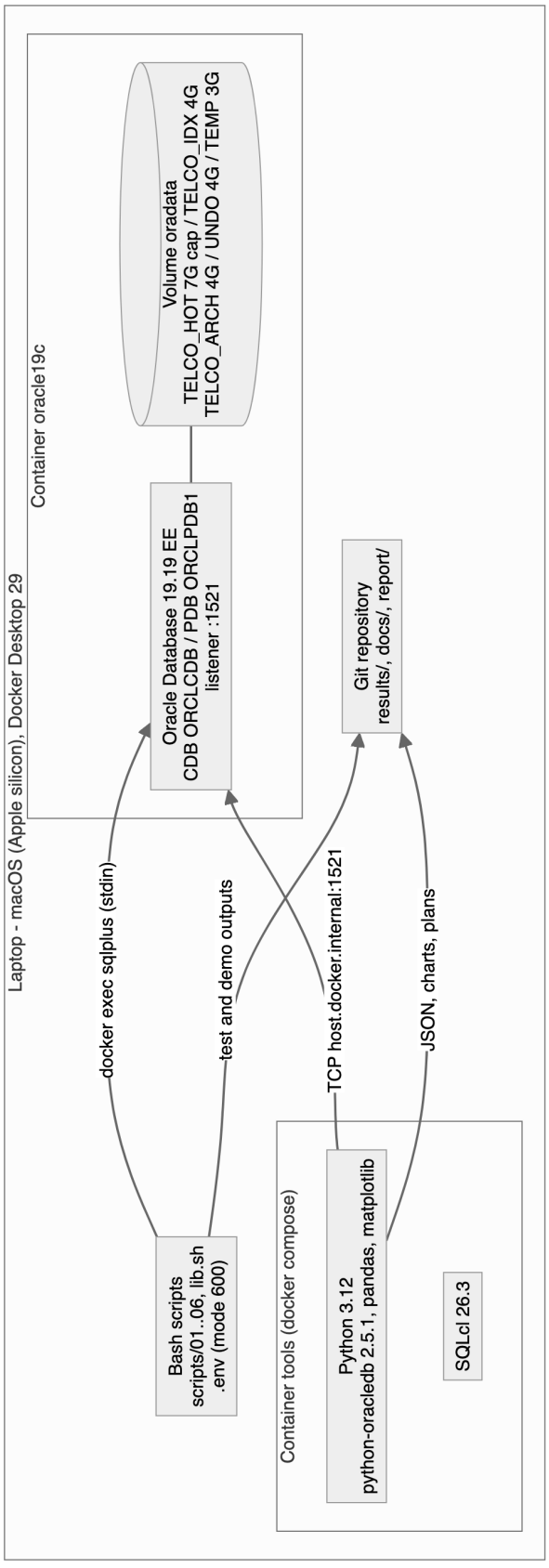


Figure 6.1: Deployment diagram

6.3 Prerequisites and installation

The repository README gives the full procedure, including how to rebuild the Oracle container from Oracle's official Docker build files. With the container running, the whole solution is installed by five scripts:

```
scripts/init_env.sh          # .env with random passwords (git-ignored, mode 600)
scripts/01_setup_db.sh       # tablespaces with caps, accounts            [SYSDBA]
scripts/02_create_schema.sh  # tables, views, PL/SQL packages             [TELCO_OWNER]
scripts/03_load_data.sh      # 23.76 M rows over 72 months (about 6 min)
scripts/05_benchmark.sh      # benchmark + report (about 25 min)
```

A smoke configuration (3 000 rows per month) runs the same chain in about one minute and is used for the destructive functional and security tests (`scripts/04_run_tests.sh`).

`scripts/07_dashboard.sh` starts the web dashboard (Chapter 8) as a third Compose service built from the tools image. It listens on `http://localhost:8501` only, and reads the account passwords from `.env` without ever displaying them.

7 Solution Implementation

7.1 Repository organisation

```
scripts/       numbered entry points (setup, schema, load, tests, benchmark, demo, reset)
sql/00_admin   tablespaces, accounts, teardown                        (SYSDBA)
sql/01_schema  DLM infrastructure, reference data, billing tables,
               data generator, DLM_PKG, BASELINE_PKG, grants        (TELCO_OWNER)
sql/02_data    history load, partitioned copy
sql/03_tests   functional and security tests (assertions, exit code)
sql/04_demo    demonstration steps
python/bench   benchmark harness and report
results/       benchmark JSON, summary, charts, execution plans, test and demo outputs
docs/, report/ documentation, diagrams and this report
```

All scripts share `scripts/lib.sh`. It loads `.env`, runs SQL*Plus inside the database container with `WHENEVER SQLERROR EXIT FAILURE` and `SET VERIFY OFF`, and pipes the SQL and the connection string through standard input, so no password appears on a command line.

7.2 Data generation

The history is generated inside the database with set-based SQL: one direct-path `INSERT /*+ APPEND */` per month. It produces 23.76 million rows in about three minutes. `ORA_HASH(n, buckets, seed)` replaces random functions, so the dataset, and therefore every benchmark, is identical from one run to the next. The distribution is realistic: 50 % data, 30 % voice, 15 % SMS and 5 % roaming. Volume grows linearly from 70 % to 130 % of the average month. Recent months contain unpaid bills, and old months are settled.

Listing 7.1: Generator core (excerpt of `04_datagen.sql`)

```
select n,
       #FIRST_ID# + n                                        as record_id,
       ora_hash(n, #CUSTS# - 1, #SEED#) + 1                  as customer_id,
       #MONTH# + ora_hash(n, #SECS#, #SEED# + 1) / 86400     as bill_date,
       case when h < 50 then 'DATA' when h < 80 then 'VOICE'
            when h < 95 then 'SMS'  else 'ROAMING' end       as service_type,
       ...
from (select level as n, ora_hash(level, 99, #SEED# + 6) as h
      from dual connect by level <= #ROWS#)
```

The same history is first loaded into the baseline heap table, with its three indexes, and then copied by one direct-path insert into the partitioned table, where interval partitioning creates the 72 monthly partitions. At load time, `capture_control_totals` records the row count, the amount total and the sum of a fingerprint of every column for each month. A first validation proves that the partitioned copy matches the baseline exactly (72/72 months).

7.3 Partitioned schema

Listing 7.2: Partitioned and archive tables (excerpt of `03_billing_tables.sql`)

```
create table billing_records
  tablespace telco_hot
  partition by range (bill_date) interval (numtoyminterval(1, 'MONTH'))
  ( partition p_initial values less than (date '2020-11-01') )
  as select * from billing_records_base where 1 = 0;

create table billing_archive
  tablespace telco_arch
  row store compress basic
  partition by range (bill_date) interval (numtoyminterval(1, 'MONTH'))
  ( partition p_initial values less than (date '2020-11-01') )
  as select * from billing_records_base where 1 = 0;
```

Interval partitions get system names (`SYS_P123`). `dlm_pkg.rename_partitions` evaluates each partition's `HIGH_VALUE` and renames the table partitions, and the matching local index partitions (matched by position), to `P_YYYYMM`.

7.4 The lifecycle package `DLM_PKG`

Every operation works on one month, refuses what the policy protects, logs its result in an autonomous transaction and can be re-run safely. Partition DDL first sets `ddl_lock_timeout = 60`, so it waits for the application's in-flight transactions instead of failing at once.

7.4.1 WARM: freeze, capture, move

Listing 7.3: Core of `warm_partition`

```
-- 1. Freeze FIRST: READ ONLY waits for in-flight transactions on the month
--    and blocks new ones, so nothing can change between capture and move.
set_read_only(c_main, l_part, true);
if not (l_ts = 'TELCO_ARCH' and l_comp = 'ENABLED') then
  -- 2. The frozen content becomes the integrity reference.
  capture_month_totals(c_main, l_month);
  -- 3. MOVE is allowed on a read-only partition (rows are not changed).
  execute immediate 'alter table ' || c_main || ' move partition ' || l_part ||
                    ' tablespace telco_arch row store compress basic update global indexes';

end if;
-- 4. Rebuild the local index partition in TELCO_ARCH (also completes an
--    interrupted warm); index partitions are matched by position, not by name.
rebuild_local_indexes(l_part);
```

The order of these steps came out of a review. An earlier version captured the totals before setting the partition read-only, so a bill committed between the two steps made every later validation fail. We verified on the database that `MOVE PARTITION` is allowed on a read-only partition, while `DROP PARTITION` is not (ORA-14466); this allowed the freeze-first order.

7.4.2 ARCHIVE: two exchanges, resumable

Listing 7.4: Core of `archive_partition` (simplified)

```
-- step 0: rows left in BILLING_XCHG by an interrupted run? -> resume at step 2
if not l_resumed then
  if not is_warm(l_month) then warm_partition(l_month, p_run_id, l_as_of); end if;
  l_rows := count_partition(c_main, l_part);
  set_read_only(c_main, l_part, false);
  execute immediate 'alter table billing_records exchange partition ' || l_part ||
    ' with table billing_xchg including indexes without validation update global indexes';
end if;
execute immediate 'lock table billing_archive partition for (' || date_literal(l_month)
                  || ') in share mode';                    -- creates the partition
execute immediate 'alter table billing_archive exchange partition ' || l_apart ||
                  ' with table billing_xchg including indexes without validation';
if count_partition(c_archive, l_apart) <> l_rows then
  raise_application_error(-20030, 'Row count mismatch ...');
end if;
set_read_only(c_archive, l_apart, true);
-- step 3: drop the now empty online partition (update global indexes)
```

The procedure refuses any month still inside the 36-month online window. It warms a month that is not yet warm, so the archive is always compressed. It never truncates the staging table: if `BILLING_XCHG` holds rows of a different month, it stops and asks for a manual check.

7.4.3 PURGE and guards

`purge_partition` drops the month's partition in the archive (or in the online table if it was never archived). It refuses (`ORA-20010`, logged as `REFUSED`) any month inside the retention period, and any requested retention shorter than `MIN_RETENTION_MONTHS`, even if an operator passes another value.

7.4.4 Integrity validation

Listing 7.5: Row fingerprint used by the control totals

```
ora_hash(record_id || '|' || invoice_id || '|' || customer_id || '|' || plan_id || '|' ||
         to_char(bill_date, 'YYYYMMDDHH24MISS') || '|' || service_type || '|' ||
         region_code || '|' || duration_sec || '|' || to_char(volume_mb, 'FM99999990.00')
    ||
         '|' || to_char(amount, 'FM99999990.00') || '|' ||
         to_char(tax_amount, 'FM99999990.00') || '|' || payment_status)
```

`validate_integrity` compares, month by month, the row count, the amount total and the sum of these fingerprints in a view (`BILLING_ALL`) with the control totals. Months before the retention cut must be absent. Hot months are live, so the monthly job validates frozen months only; their totals are re-captured at the moment they are frozen.

7.5 Application write window

Listing 7.6: View recreated by `refresh_app_window` (as of 2026-11)

```
create or replace view billing_current_v as
  select * from billing_records
  where bill_date >= date '2025-11-01' and bill_date < date '2027-01-01'
  with check option constraint billing_current_v_ck;
```

The application has `INSERT` and `READ` only on this view. A late row dated in an archived month, or far in the future, is rejected. During testing we found that Oracle still creates the (empty) interval partition for a rejected row, because partition creation is autonomous. The monthly job therefore drops empty partitions outside the window.

7.6 Baseline package

`BASELINE_PKG` reproduces the client's current method with the same policy, the same guards and the same operation order: archive = `INSERT INTO ... SELECT` followed by `DELETE`, and purge = `DELETE`. The comparison therefore measures the technique, not different rules.

7.7 Benchmark harness

`python/bench` connects as the owner from the tools container. Around every measured operation it reads `V$MYSTAT` (logical reads, physical reads, redo, undo, CPU) and the elapsed time. For queries it fetches the execution plan with `DBMS_XPLAN.DISPLAY_CURSOR` using the cursor's `prev_sql_id`. Each query is run once to warm up and five times measured, with identical SQL and binds on both designs. The run order is: queries on the full history; two monthly loads with statistics refreshes; lifecycle of both designs; queries again; storage and integrity. Results are saved after every operation to `results/benchmark_results.json`, and `bench.report` produces the summary and the charts.

8 Practical Usage Examples

The following scenarios were executed on the full dataset (23.76 million rows) during the demonstration rehearsal. The outputs below are copied from `results/demo/`, with blank lines and timing lines removed. Before step 1 the policy had been applied as of the previous month (2026-10), so the scenarios show the routine monthly situation.

Every scenario can also be run from a web dashboard (`scripts/07_dashboard.sh`, then `http://localhost:8501`). It is built with Streamlit and shows the tiers, the business query on both designs, the dry run, the incident and its recovery, the monthly job and the security checks, each started by one button. The dashboard calls the same `DLM_PKG` procedures as the scripts, and serialises write operations so two clicks cannot run a job twice. `scripts/07_dashboard.sh --fresh` reloads the data and prepares the demonstration state.

8.1 Scenario 1 – State of the tiers

Listing 8.1: `scripts/06_demo.sh 1` (excerpt)

```
=== BILLING_RECORDS partitions by tier ===
TIER     TABLESPACE_N COMPRESS PARTITIONS FIRST_PART LAST_PART    NUM_ROWS
-------- ------------ -------- ---------- ---------- ---------- ----------
HOT      TELCO_HOT    DISABLED         13 P_202510   P_202610      5359479
WARM     TELCO_ARCH   ENABLED          24 P_202310   P_202509      8656225
=== BILLING_ARCHIVE partitions ===
PARTITIONS FIRST_PART LAST_PART    NUM_ROWS
---------- ---------- ---------- ----------
        24 P_202110   P_202309      7049916
```

Each tier is in the expected tablespace and format. Retained history is split between the online table (hot + warm) and the archive.

8.2 Scenario 2 – Month-end revenue report (business scenario)

Finance asks for October 2026 revenue by service. The same statement runs on both designs and returns identical results (4 rows: 128 397 voice, 64 382 SMS, 214 795 data and 21 426 roaming lines).

Listing 8.2: Execution plans of the same query on both designs

```
-- Baseline (heap table)
| Id  | Operation          | Name                | Rows  |
|   0 | SELECT STATEMENT   |                     |       |
|   1 |  HASH GROUP BY     |                     |     4 |
|   2 |   TABLE ACCESS FULL| BILLING_RECORDS_BASE |   336K|

-- Partitioned design
| Id  | Operation               | Name            | Rows  | Pstart| Pstop |
|   0 | SELECT STATEMENT        |                 |       |       |       |
|   1 |  HASH GROUP BY          |                 |     4 |       |       |
|   2 |   PARTITION RANGE SINGLE|                 |   429K|    73 |    73 |
|   3 |    TABLE ACCESS FULL    | BILLING_RECORDS |   429K|    73 |    73 |
```

The baseline must read the whole six-year table. One month is about 1.8 % of the rows, still too many for the `bill_date` index. The partitioned plan reads one partition. For interval partitioning, Pstart/Pstop is the interval number counted from the first partition (month 73), not the partition's current position.

8.3 Scenario 3 – Dry run of the monthly job

Listing 8.3: `exec dlm_pkg.run_lifecycle(p_dry_run => true)` for 2026-11

```
ACTION       MONTHS FIRST_MONTH   LAST_MONTH
-------- ---------- ------------- ------------
PURGE             1 2021-10       2021-10
WARM              1 2025-10       2025-10
ARCHIVE           1 2023-10       2023-10
```

The operator can check exactly what the job will do before anything changes.

8.4 Scenario 4 – Incident: archive job crashes, then recovers

We simulate a crash right after the first exchange of the archive of 2023-10 (`p_simulate_failure => 'AFTER_STEP1'`).

Listing 8.4: Incident and recovery (`scripts/06_demo.sh 4`, 16 s in total)

```
=== Rows of 2023-10 before the incident ===
     328606
=== Archive job crashes right after step 1 (exchange out) ===
ORA-20020: Simulated failure after step 1
=== Where is the data? Not lost: it waits in BILLING_XCHG ===
ROWS_IN_EXCHANGE_TABLE      328606
=== Integrity check (2021-10 .. 2023-10) detects the incomplete month ===
MONTHS_IN_ERROR             1
=== Recovery: simply re-run the job ===
PL/SQL procedure successfully completed.
ROWS_ARCHIVED               328606
ROWS_IN_EXCHANGE_TABLE      0
MONTHS_IN_ERROR             0
=== Audit trail ===
ACTION    TARGET       STATUS   MESSAGE
VALIDATE  BILLING_ALL  OK       25/25 months match control totals (from 2021-10 to 2023-10)
ARCHIVE   P_202310     OK       resumed after interruption; exchanged into BILLING_ARCHIVE, read only
ARCHIVE   2023-10      RESUMED  found staged rows from an interrupted run - resuming at step 2
VALIDATE  BILLING_ALL  ERROR    24/25 months match control totals (from 2021-10 to 2023-10)
```

```
ARCHIVE   P_202310     ERROR    SIMULATED FAILURE after step 1 (data is safe in BILLING_XCHG)
```

No row was lost. The integrity check detected the incomplete month, a simple re-run resumed at step 2, and the audit trail records each event.

8.5 Scenario 5 – Guard and monthly job

Listing 8.5: Guard, then the monthly job for 2026-11 (46 s)

```
SQL> exec dlm_pkg.purge_partition(date '2022-01-01', p_retention_months => 24)
ORA-20010: PURGE 2022-01 refused: requested retention 24 months is below the
           legal floor of 60 months

SQL> exec dlm_pkg.run_lifecycle
ACTION     STATUS     OPERATIONS  TOTAL_SEC ROWS_PROCESSED
---------- ---------- ---------- ---------- --------------
PURGE      OK                  1        .07         261676
WARM       OK                  1       3.22         395535
VALIDATE   OK                  1      15.57
RUN_END    OK                  1      45.51
MESSAGE: 48/48 months match control totals (from 2021-11 to 2025-10)

=== Archived history is still one query away (BILLING_ALL) ===
DISPUTED_2022_03        2740
```

Purging a month of 261 676 rows took 0.07 s. Compressing one month took 3.2 s. Most of the 46-second run is statistics and the validation of the 48 frozen months (16 s).

8.6 Scenario 6 – What the billing application cannot do

Listing 8.6: Asserted security checks as `TELCO_APP` (excerpt)

```
PASS  S02 insert a bill in a HOT month (rolled back)
PASS  S03 late insert dated in an archived month (2022-03) rejected  (ORA-1402)
PASS  S05 direct write into the base table rejected  (ORA-942)
PASS  S06 delete billing records rejected  (ORA-1031)
PASS  S09 change retention policy rejected  (ORA-942)
PASS  S10 run a lifecycle job rejected  (ORA-6550)
PASS  S14 lock of reference data rejected (READ grant)  (ORA-1031)
Security checks (TELCO_APP) passed: 14   failed: 0
```

9 Testing and Validation

9.1 Methodology

We used three levels of tests:

1. **Automated functional tests** (`sql/03_tests/01_functional_tests.sql`): 43 checks in PL/SQL. Each prints `PASS` or `FAIL`, and the script exits non-zero if any check fails. The tests apply the whole lifecycle, so they run on a freshly loaded *smoke* dataset (216 000 rows, same calendar). A precondition refuses to run them on a dataset larger than 2 million rows, so the benchmark data cannot be destroyed by mistake.
2. **Asserted security tests**: 25 checks run as `TELCO_APP` (14) and `TELCO_ANALYST` (11). Each expects a precise result (success or a given Oracle error code).
3. **Full-scale validation**: the benchmark (Chapter 10) and the demonstration rehearsal (Chapter 8) on 23.76 million rows, both ending with an integrity validation.

Running `scripts/04_run_tests.sh` deletes earlier outputs, then writes new ones to `results/tests/`.

9.2 Functional test scenarios

Table 9.1: Functional tests (all 43 pass)

Tests	What is verified
T01–T02	Partitioned copy equals the baseline (control totals); one partition per month.
T03–T06	Dry run plans 12 purges, 48 warm moves and 24 archives, and changes nothing (partitions, tablespaces, compression, read-only).
T07–T11, T37–T39	Guards: purge below the legal floor, purge inside retention, archive inside the online window, warm of a hot month; the last three are also tested at their exact boundary month; all refusals logged.
T12–T17	Crash after exchange step 1: no row lost (data in `BILLING_XCHG`), integrity check detects it, re-run resumes and completes, logged as `RESUMED`, integrity restored.
T34–T35	Warm of a system-named partition rebuilds its local index (matched by position); a warm interrupted after the move is completed by a re-run.
T18–T24, T42	Full run: 36 online months, 24 archived, 24 warm months compressed, read-only and in `TELCO_ARCH`, 12 hot months untouched, nothing older than retention, integrity, all indexes usable, all archive partitions compressed and read-only.
T25	Second run is a no-op (no operation performed, same partitions, integrity).
T26	Archived data cannot be modified (ORA-14466).

Tests	What is verified
T27–T31	Late and far-future inserts rejected by the window; only empty partitions remain; an insert in a hot month is accepted; the next run removes the empty partitions.
T32	The monthly job still succeeds after the application commits new rows in a hot month.
T41	A crashed archive is resumed automatically by the monthly job itself.
T36	`run_lifecycle(p_as_of => 2026-12)` applies the policy one month later, and every operation uses that month.
T40	A deliberately altered frozen month stops the monthly job (ORA-20040).
T33	The baseline lifecycle archives 24 months and keeps every retained row.

9.3 Security test scenarios

Table 9.2: Security checks (all 25 pass)

Account	Verified
`TELCO_APP`	**Allowed:** read the window, insert in a hot month. **Rejected:** inserts in an archived month, in the first non-hot month (window boundary), far in the future or just after the window; direct writes to the base table; delete; update; reading the archive table; changing the retention policy; running lifecycle procedures; DDL; locking reference data.
`TELCO_ANALYST`	**Allowed:** read the history through `BILLING_ALL`. **Rejected:** insert, update or delete through the view; `LOCK TABLE` and `SELECT ... FOR UPDATE` (the `READ` grant); direct access to the base tables, the archive, the application window or the policy; running lifecycle procedures.

9.4 Verification of the acceptance criteria

Table 9.3: Acceptance criteria

Criterion	Status	Evidence
The design preserves data integrity	met	T01, T17, T23, T31, T33, T40; 62/62 months after the benchmark
Retention operations work correctly	met	T03–T42, job log, demonstration
Improvements measured on comparable workloads	met	Chapter 10
Licensing and trade-offs documented	met	Chapters 4, 12

10 Performance and Reliability Evaluation

10.1 Measurement protocol

- Same database, same 23.76 million rows, same SQL text and bind values, same retention policy and operation order for both designs.
- Each query: one warm-up run and five measured runs; medians are reported. Logical reads (LIO), physical reads, redo, undo and CPU come from `V$MYSTAT` deltas, and plans are saved in `results/plans/`.
- **LIO is the primary indicator.** Full scans of the 1.6 GB baseline table use serial direct-path reads, which bypass the buffer cache. Every baseline full scan therefore reads about 204 000 blocks from disk, and its elapsed time varies with the laptop's I/O (from 0.5 to 2.6 s over five runs in one case). Partitions stay in cache. Elapsed times are client-side, so sub-millisecond queries are mostly noise.

10.2 Query performance

Table 10.1: Workload results on the full 72-month history (before lifecycle), medians

	Query	Elapsed (ms)		Logical reads		Gain
		Base	Part.	Base	Part.	time / LIO
Q1	Monthly revenue by service	535.3	37.1	204 092	3 746	14× / 54×
Q2	Customer history, 3 months	0.9	0.8	6	12	1.1× / 0.5×
Q3	Annual revenue 2025	1 108.5	580.8	204 092	39 787	1.9× / 5.1×
Q4	Open receivables, 6 months	618.5	176.9	204 092	21 829	3.5× / 9.3×
Q5	Lookup by record id	0.6	0.6	4	4	1.0× / 1.0×
Q6	Revenue per year, no filter	2 623.3	2 750.7	204 092	204 158	1.0× / 1.0×
Q7	Disputes in March 2022	1 410.7	13.0	204 108	2 369	109× / 86×

Table 10.2: Workload results after the retention policy (Q7 through the online + archive views)

	Query	Elapsed (ms)		Logical reads		Gain
		Base	Part.	Base	Part.	time / LIO
Q1	Monthly revenue by service	534.1	39.5	211 566	3 746	14× / 56×
Q2	Customer history, 3 months	1.2	0.8	6	12	1.5× / 0.5×
Q3	Annual revenue 2025	978.9	664.3	211 566	30 338	1.5× / 7.0×
Q4	Open receivables, 6 months	595.2	174.8	211 566	21 829	3.4× / 9.7×
Q5	Lookup by record id	0.4	0.4	4	4	1.0× / 1.0×
Q6	Revenue per year, no filter	2 089.7	1 636.2	211 566	103 677	1.3× / 2.0×
Q7	Disputes in March 2022	383.0	8.9	163 098	1 690	43× / 97×

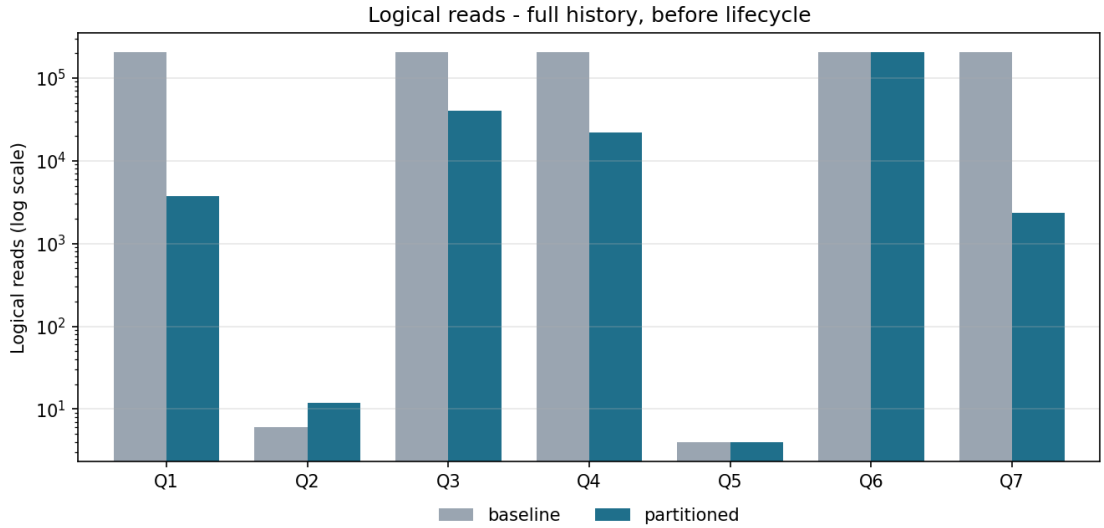


Figure 10.1: Logical reads per query on the full history (log scale)

Interpretation.

- **Date-bounded reports** (Q1, Q4, Q7) gain the most, because partition pruning reads only the requested months. The baseline full-scans the table, since a single month is too large a share of the rows for its `bill_date` index.
- **Q3** reads 5–7 times fewer blocks but runs only 1.5–1.9 times faster. It aggregates about 4.6 million rows, so CPU dominates once I/O is gone. After the lifecycle, 10 of its 12 months are compressed, and decompression costs CPU.
- **Q6 has no date predicate**, so nothing can be pruned. Partitioning is not a universal accelerator. After the lifecycle Q6 reads half the blocks, because the online table holds 38 months, 24 of them compressed.
- **OLTP lookups** are unchanged in time. Q5 uses the global primary key in both designs. Q2 reads 12 blocks instead of 6, because the local index has one B-tree per month and the query covers three months. This is the cost of local indexes, and it is negligible here.

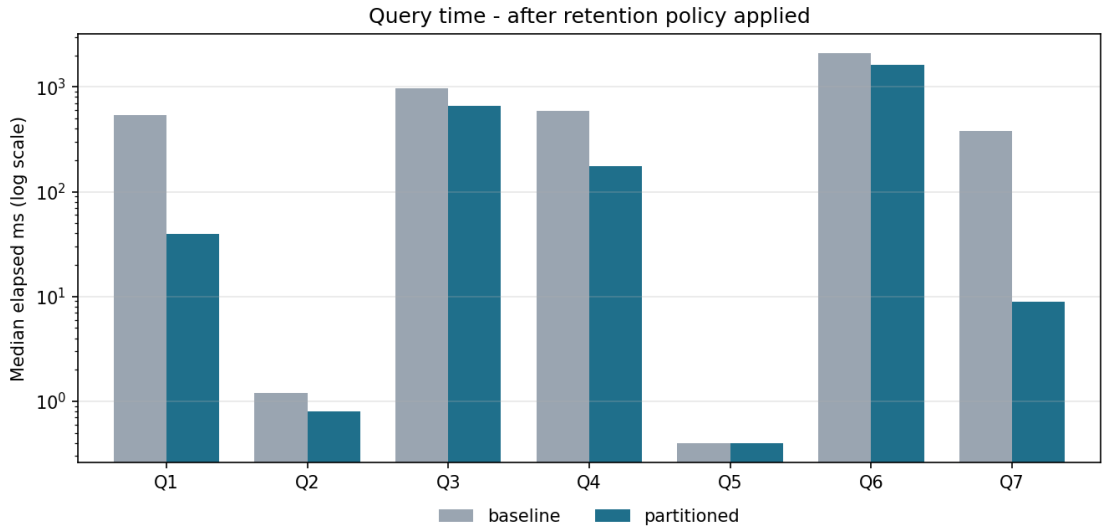


Figure 10.2: Median elapsed time per query after the retention policy (log scale)

10.3 Maintenance operations

The lifecycle was applied in steady-state order: the oldest months were warmed and archived first, then purged *from the archive*, exactly as they would be after two years in the archive. Both designs followed this order.

Table 10.3: Maintenance cost per operation (averages)

Operation (one month)	Seconds		Redo (MB)		Gain
	Base	Part.	Base	Part.	time
Archive	26.17	0.42	498.8	6.3	62×
Purge	5.14	0.02	110.0	<0.1	≈250×
Warm (freeze, move, compress)	–	3.06	–	152.9	new step
Monthly job	**31.31**	**3.51**	**608.8**	**159.3**	**8.9×**
Load one month (429 000 rows)	4.36	2.13	101.9	76.1	2.0×
Statistics, routine refresh	35.44	14.44	5.7	2.6	2.5×
Statistics, first after migration	32.21	75.27	5.6	94.3	0.4×

The *monthly job* is the fair steady-state comparison. In the baseline it is one archive and one purge. In the partitioned design it is one warm, one archive and one purge, because the zero-copy archive relies on the month having been compressed by the warm step. It is 8.9 times faster, with 3.8 times less redo and 4.0 times less undo. Applying the whole policy to the six-year history at once took 1 004 s and 19.3 GB of redo for the baseline, against 199 s and 9.4 GB for the partitioned design. Of the partitioned figures, 184 s and 9.2 GB are the one-time warm catch-up of 60 months.

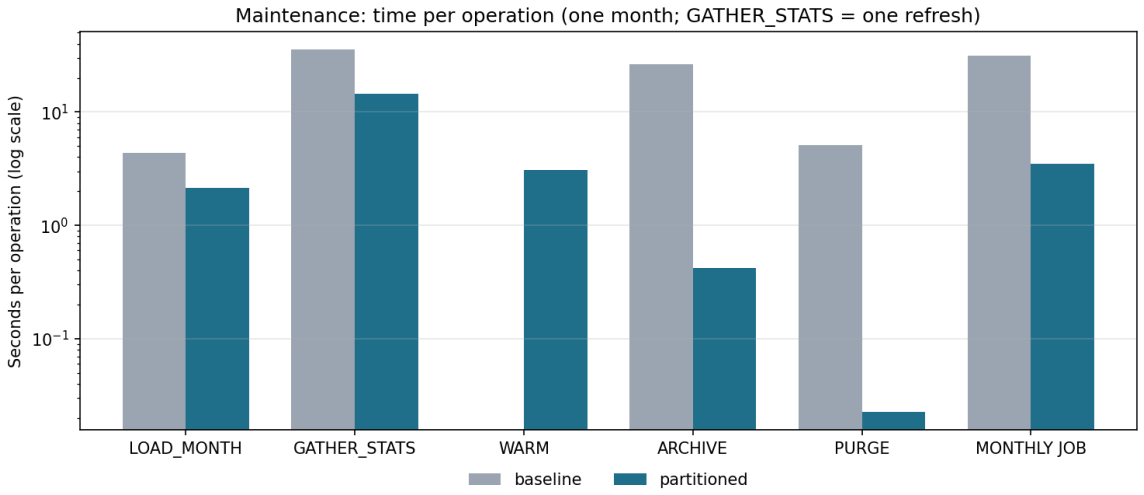


Figure 10.3: Maintenance time per operation (log scale)

Interpretation.

- Archive and purge become metadata operations: their cost no longer depends on the number of rows. The baseline's `DELETE`s generate undo and redo for every row and every index entry. In ARCHIVELOG mode (production) this redo would also have to be archived, backed up and shipped to a standby.
- **WARM is the main cost of the new design** (3.1 s and 153 MB of redo per month). It moves and compresses the month, maintains the global primary key, rebuilds the local index par-

tition and re-captures the control totals. We did not measure these parts separately; global index maintenance is the likely main part.

- The **first statistics refresh after the migration** re-analysed all 73 partitions (75 s). The next routine refresh re-analysed only the new partition (14 s, 2.5 times faster than the baseline). This is a one-time cost of the migration.

10.4 Storage

Table 10.4: Space used, tables and their indexes (MB)

Table	Before	After policy
Baseline online (`BILLING_RECORDS_BASE`)	3 152	3 248
Baseline archive (`BILLING_ARCHIVE_BASE`, 2 indexes)	0	1 304
Partitioned online (`BILLING_RECORDS`)	3 121	1 975
Partitioned archive (`BILLING_ARCHIVE`, 1 local index)	0	587
Retained data, baseline / partitioned		**4 552 / 2 562 (-44 %)**

After 36 of its 74 months were moved out, the baseline table did not shrink. It even grew, because new months were loaded above the high-water mark, and full scans still read about 212 000 blocks. The baseline archive also keeps the space of the months deleted from it. Releasing that space would require `SHRINK SPACE` or `MOVE` on large tables. With partitions, a purge releases space immediately and BASIC compression shrinks warm and archived months. The archive comparison combines compression, one index fewer and the space the baseline did not release.

10.5 Integrity and reliability

After the whole benchmark, **62 of 62** retained months match their control totals in both designs (row count, amount total and fingerprint sum), and every purged month is absent. Together with the functional tests (crash and resume, interrupted warm, deliberate corruption detected) this supports the reliability indicators K8 and K9.

10.6 Indicators against targets

Table 10.5: KPI results

KPI	Target	Result	Status
K1	$\geq$5$\times$ time, $\geq$10$\times$ LIO	14$\times$ / 54–56$\times$	met
K2	$\geq$5$\times$ fewer LIO	86$\times$ before, 97$\times$ after	met
K3	no time regression $>$2$\times$	Q5 unchanged; Q2 0.9 $\rightarrow$ 0.8 ms	met
K4	archive $\geq$10$\times$, $<$5 s	archive operation 62$\times$, 0.42 s; whole monthly job 8.9$\times$, 3.5 s	met / **partly**
K5	purge $<$1 s	0.02 s	met

KPI	Target	Result	Status
K6	load + stats not slower	2.0× and 2.5× faster (first gather slower, once)	met
K7	≥25 % less storage	−44 %	met
K8	100 % integrity	62/62 months, both designs	met
K9	0 rows lost, re-run recovers	T12–T17, T41, Scenario 4	met
K10	100 % security checks	25/25	met
K11	one command per month	`dlm_pkg.run_lifecycle`	met

Ten indicators are met. K4 is met for the archive operation itself (62 times faster). When the warm step that the archive depends on is included, the whole monthly job is 8.9 times faster, just under the 10× target. Section 12.3 discusses how to reduce the warm cost.

11 Challenges Encountered and Solutions

The work went through four review rounds. The last ones used independent reviewers who re-read the code without our conclusions; every finding was checked against the code, and on the database when Oracle's behaviour was uncertain. The main problems and their solutions follow. The full list is in `docs/work_log.md`.

Table 11.1: Main problems and corrective actions

Problem	Cause / analysis	Solution
Laptop disk almost full; UNDO and TEMP could grow to 32 GB each	default autoextend of the course container	caps on every file, cleanup of unused images, free-space checks in the scripts
`sqlplus / as sysdba` failed (ORA-12162)	`ORACLE_SID` not set in `docker exec` shells	SID passed explicitly by `lib.sh`
Partitions named `SYS_Pnnn`	interval partitioning generates names	rename from `HIGH_VALUE`; local index partitions matched by *position* (their system names differ from the table's)
The application could insert a row dated in an archived month; even after `ROLLBACK` an empty partition remained online	interval partition creation is autonomous	application write window (view `WITH CHECK OPTION`); the monthly job drops empty partitions outside the window
Archive interrupted between the two exchanges	two DDL steps cannot be one transaction	staging table never truncated; next run detects staged rows and resumes; tested by a simulated crash
A bill committed between the capture of a month's totals and its move made every later run fail	order of the warm steps	freeze first (READ ONLY waits for in-flight transactions), then capture, then move (verified: MOVE is allowed on a read-only partition)
Read-only accounts could lock the tables and block the job	`SELECT` allows `LOCK TABLE` (verified on the DB)	`READ` grants; DDL waits up to 60 s for in-flight DML
Passwords could appear in clear text	substitution into PL/SQL literals (visible in `V$SQL`); SQL*Plus echoes a failing line	users created without a password inside PL/SQL, then passwords set by plain `ALTER USER`; no statement can fail
First statistics refresh slower than the baseline (75 s)	all partitions seen as stale once after the 24 M-row migration	measured separately from the routine refresh (14 s); documented as a one-time cost
Purge first measured on the online table	benchmark order did not match steady state	warm and archive first, then purge from the archive, for both designs

Problem	Cause / analysis	Solution
Archive KPI excluded the warm step it depends on	the KPI compared one operation only	the whole monthly job is reported as well (8.9×)
Elapsed times of baseline full scans varied widely (0.5–2.6 s for the same query)	direct-path reads always go to disk	logical reads used as the primary indicator, medians of 5 runs

12 Discussion and Future Improvements

12.1 Strengths

- **Measured, not claimed.** Every gain comes from the same data and workload on both designs, with plans, raw JSON and test outputs in the repository.
- **Retention as metadata.** Archive and purge no longer depend on data volume. Their cost stays flat as the business grows, while the baseline grows linearly.
- **Safety by design.** Guards refuse illegal purges, frozen data is read-only, the application can only write the hot window, every operation is logged and resumable, and integrity is proved against control totals. The job stops if a frozen month changes.
- **Operations.** One command per month, a dry-run mode, the policy stored as data, and readable partition names.

12.2 Limitations and remaining risks

- **Licence.** Partitioning is a paid Enterprise Edition option. On Standard Edition 2 the design would fall back to one table per month behind a `UNION ALL` view, without basic compression (EE only) and with weaker pruning.
- **Queries without a date filter** gain nothing (Q6), and multi-month lookups through local indexes read a few more blocks (Q2).
- **Warm cost.** It is probably dominated by maintenance of the global primary key (not measured separately); the monthly job is 8.9× faster, not the 10× we targeted.
- **No `record_id` index on archived rows**: a lookup by `record_id` through `BILLING_ALL` scans the archive partitions.
- **Concurrency.** The application can still hold locks (its `INSERT` grant allows `LOCK TABLE`), and two lifecycle runs are not serialised. Guards and row counts prevent data loss, but a run can fail and must be re-run.
- **Evaluation context.** One laptop, single-user measurements, synthetic data (uniform customers, no corrections of old bills) and 24 million rows. Absolute times are indicative; logical reads and redo are the reliable comparisons.
- **Owner privileges.** `TELCO_OWNER` can change the policy and read PDB-wide `V$SQL`. In production it would be locked, and the job would run under a scheduler with auditing on `DLM_CONFIG`.

12.3 Possible improvements

1. **Reduce the warm cost:** use `NOLOGGING` moves followed by a backup (only without a standby), warm several months in parallel off-peak, or switch to a local primary key on `(record_id, bill_date)` if the application can accept it.
2. **Schedule and serialise** the monthly job with `DBMS_SCHEDULER` and a `DBMS_LOCK` lock, with alerting on `ERROR` or `REFUSED` log entries.

3. **Archive statistics:** gather table-level synopses on `BILLING_XCHG` before each exchange, so exchanged partitions arrive with incremental synopses.
4. **Tiered storage:** place `TELCO_ARCH` on cheaper storage and make archive tablespaces read-only, so backups of history are taken once.
5. **Measure under concurrency:** run the application workload during the lifecycle job, and test on production-scale volumes.
6. **Audit:** Unified Auditing policies on `DLM_CONFIG` and on lifecycle procedures.

13 General Conclusion

The client's billing history had become a single, ever-growing table: reports were slow, retention was an expensive `DELETE`, and nothing proved that billing data stayed intact. We replaced this with a monthly partitioned design and a lifecycle package that moves each month through four tiers: hot, warm (compressed and read-only), archive and purge. Every step is guarded, logged, resumable and validated.

The client's objectives are met. Date-bounded reports read up to 97 times fewer blocks, and single-month reports run 14 to 43 times faster. Archiving a month takes 0.4 s instead of 26 s, purging one 0.02 s instead of 5 s, and the whole monthly job is 8.9 times faster with 3.8 times less redo. Retained data uses 44 % less storage. All retained months match their control totals after the whole policy. The solution passes 43 functional tests and 25 security checks, including a crash in the middle of an archive, late inserts into old months and a deliberately corrupted month, which the job detects and refuses to accept.

The project also showed where partitioning does not help. Queries without a date filter gain nothing, local indexes add a few reads to multi-month lookups, and the global primary key makes compression the main cost of the new design. These trade-offs, the licensing constraints and the limits of a laptop evaluation are documented so the DBA Manager can decide with full information.

Beyond the numbers, the main lesson is method. Each claim is backed by a measurement or a test, each result can be reproduced from the repository, and each review round found real defects that we corrected before reporting the results.

References

[1] Oracle, *Oracle Database VLDB and Partitioning Guide, 19c* – interval partitioning, partition exchange, read-only partitions, partition maintenance and global index maintenance.

[2] Oracle, *Oracle Database SQL Language Reference, 19c* – `ALTER TABLE ... MOVE / EXCHANGE / DROP PARTITION`, `LOCK TABLE`, `CREATE VIEW ... WITH CHECK OPTION`, `READ` object privilege.

[3] Oracle, *Oracle Database SQL Tuning Guide, 19c* – incremental statistics, partition pruning, `DBMS_XPLAN`.

[4] Oracle, *Oracle Database PL/SQL Packages and Types Reference, 19c* – `DBMS_STATS`, `DBMS_XPLAN`, `DBMS_UTILITY`, `DBMS_ASSERT`.

[5] Oracle, *Oracle Database Licensing Information User Manual, 19c* – Partitioning, Advanced Compression, Diagnostics and Tuning Packs, basic table compression.

[6] Oracle, *Oracle Database Reference, 19c* – `V$MYSTAT`, `V$STATNAME`, `V$SESSION`, `V$SQL`, `USER_TAB_PARTITIONS`.

[7] Oracle, *docker-images* repository, `OracleDatabase/SingleInstance` build files (`https://github.com/oracle/docker-images`).

[8] *python-oracledb documentation*, Oracle (`https://python-oracledb.readthedocs.io`).

[9] Course material: *General Project Guidelines* and *Project 22* specification (`docs/assignment/`).

Use of AI tools. An AI assistant (Claude) was used to help design and write scripts and documentation and to run code reviews. All code was executed and validated by the automated tests and the benchmark in the repository, and the team is responsible for the content of this report.

A Team Organisation and Schedule

A.1 Four-week plan

Table A.1: Project schedule

Week	Activities	Outcome
1	Requirements, environment assessment, technology comparison, architecture, indicators	validated design, prepared environment (tablespace caps, accounts)
2	Schema, data generator, partitioned design, lifecycle and baseline packages	first working version
3	Functional and security tests, incident simulation, benchmark, review rounds and corrections	verifiable results
4	Final benchmark, demonstration rehearsal, report and presentation	final solution and defense

A.2 Distribution of responsibilities

Both members can explain the whole solution. Each area has an owner, who presents it and answers first, and a reviewer.

Table A.2: Responsibilities

Area	Owner	Reviewer
Requirements, client scenario, indicators, technology assessment	Mohamed Baounna	[Second team member]
Architecture, partitioning and indexing design	Mohamed Baounna	[Second team member]
Lifecycle package (`DLM_PKG`)	Mohamed Baounna	[Second team member]
Environment, tablespaces, accounts, security	[Second team member]	Mohamed Baounna
Data generator, baseline design, data loading	[Second team member]	Mohamed Baounna
Functional, security and incident tests	[Second team member]	Mohamed Baounna
Benchmark harness and result analysis	Mohamed Baounna	[Second team member]
Demonstration, report and presentation	both	–

B Lifecycle Package Interface

Listing B.1: Specification of `DLM_PKG` (abridged)

```
create or replace package dlm_pkg authid definer as
  function  get_param(p_name in varchar2) return varchar2;
  function  as_of_month return date;
  procedure log_event(p_run_id, p_job, p_action, p_target, p_rows, p_elapsed,
                      p_status, p_message default null);      -- autonomous
  function  partition_month(p_table in varchar2, p_partition in varchar2) return date;
  function  partition_for(p_table in varchar2, p_month in date) return varchar2;
  procedure rename_partitions(p_table in varchar2);
  procedure warm_partition   (p_month in date, p_run_id in number default null,
                              p_as_of in date default null);
  procedure archive_partition(p_month in date, p_run_id in number default null,
                              p_simulate_failure in varchar2 default null,
                              p_as_of in date default null);
  procedure purge_partition  (p_month in date, p_run_id in number default null,
                              p_retention_months in pls_integer default null,
                              p_as_of in date default null);
  procedure refresh_app_window(p_as_of in date default null);
  procedure run_lifecycle(p_as_of in date default null, p_dry_run in boolean default false);

  procedure capture_control_totals(p_table in varchar2);
  procedure capture_month_totals(p_table in varchar2, p_month in date);
  function  validate_integrity(p_view in varchar2, p_from_month in date default null,
                               p_run_id in number default null,
                               p_to_month in date default null) return number;
end dlm_pkg;
```

Table B.1: Error codes raised by the lifecycle code

Code	Meaning
ORA-20010	operation refused by a guard (retention, online or hot window, foreign staged month)
ORA-20020	simulated failure (tests and demonstration only)
ORA-20030	row count mismatch after an archive
ORA-20040	integrity validation failed at the end of a monthly run
ORA-20098	functional tests refused (wrong calendar, dataset not fresh or too large)
ORA-20099	one or more automated tests failed

C Test Output (excerpt)

Listing C.1: `results/tests/functional_tests.txt` (excerpt)

```
PASS  T01 partitioned copy matches control totals
PASS  T03 dry run plans 12 purges
PASS  T07 purge refused when retention < legal floor (24 < 60)  (got ORA-20010 ...)
PASS  T37 purge refused at the retention boundary (2021-11)  (got ORA-20010 ...)
PASS  T12 simulated crash after exchange step 1  (got ORA-20020 ...)
PASS  T13 no row lost during the crash: rows wait in BILLING_XCHG  (2404 rows staged)
PASS  T15 re-run resumes and completes the archive
PASS  T34 warm of a system-named partition rebuilds its local index (by position)
PASS  T35 re-running an interrupted warm completes it (index rebuilt, read only)
PASS  T42 every archive partition is compressed, read-only, in TELCO_ARCH
PASS  T25 second run is a no-op (idempotent: no operation performed)
PASS  T26 archived data cannot be modified (read-only partition)  (got ORA-14466 ...)
PASS  T32 lifecycle succeeds after the application committed new HOT rows
PASS  T41 run_lifecycle resumes the crashed archive (step 0a)
PASS  T40 lifecycle stops (ORA-20040) when a frozen month was altered
PASS  T33 baseline lifecycle archives 24 months and keeps every retained row
------------------------------------------------
Tests passed: 43   failed: 0
```

D Reproducing the Results

```
git clone https://github.com/Baounna/oracle-billing-lifecycle.git && cd oracle-billing-
    lifecycle
scripts/init_env.sh
scripts/01_setup_db.sh && scripts/02_create_schema.sh && scripts/03_load_data.sh
scripts/05_benchmark.sh              # results/summary.md, results/charts/
# functional and security tests on the smoke dataset:
(umask 077; sed -e 's/^ROWS_PER_MONTH=.*/ROWS_PER_MONTH=3000/' \
                -e 's/^CUSTOMERS=.*/CUSTOMERS=5000/' .env > .env.smoke)
( export ENV_FILE=$PWD/.env.smoke; scripts/reset.sh --yes && scripts/01_setup_db.sh \
  && scripts/02_create_schema.sh && scripts/03_load_data.sh && scripts/04_run_tests.sh )
```